

UNIX & Shell Programming

Complete Question & Answer Guide — 6th Semester BCA (MAKAUT)
33 Important Questions with Standard Answers

★★★ = HIGH PRIORITY (Must Do)

★★ = MEDIUM PRIORITY (Recommended)

Unit 1 — UNIX OS, Architecture & Concepts

Q1. Define OS. Write the two parts of OS. Define multiprogramming and multitasking. Is UNIX a multitasking OS? Explain.

★★★ HIGH PRIORITY

Operating System (OS): An OS is system software that manages computer hardware and software resources and provides common services for computer programs. It acts as an interface between the user and the hardware.

Two Parts of OS:

- **Kernel** – The core part of OS that interacts directly with hardware. It manages memory, CPU scheduling, I/O, and process management.
- **Shell** – The interface between the user and the kernel. It interprets user commands and passes them to the kernel.

Multiprogramming: The technique of keeping multiple programs in memory simultaneously so that the CPU is always busy executing one of them. When one process waits for I/O, the CPU switches to another process.

Multitasking: The ability of an OS to execute more than one task (process) at the same time by rapidly switching the CPU between processes, giving the illusion of parallel execution.

Is UNIX a Multitasking OS? Yes. UNIX is a time-sharing, multitasking, and multiuser OS. It allows multiple processes to run concurrently. The CPU scheduler allocates time slices to each process. UNIX also supports multiprogramming — several jobs can be kept in memory at once. Commands like `bg`, `fg`, and `&` allow background/foreground multitasking.

Q2. Describe the layered architecture of UNIX OS. What are its advantages?

★★★ HIGH PRIORITY

Layered Architecture of UNIX: UNIX follows a layered (onion-ring) architecture:

- **Layer 1 — Hardware:** Physical components — CPU, memory, I/O devices.
- **Layer 2 — Kernel:** Core of the OS. Manages processes, memory, file system, device drivers, and system calls. Interacts directly with hardware.
- **Layer 3 — Shell:** Command interpreter. Reads user commands, interprets them, and passes to the kernel. Types: Bourne (`sh`), C shell (`csh`), Korn (`ksh`), Bash.
- **Layer 4 — Utilities/Applications:** User-level programs — editors (`vi`), compilers, mail, etc.

Advantages of Layered Architecture:

- **Portability** — UNIX can run on different hardware with minimal changes.
- **Security** — Each layer is isolated; user cannot directly access hardware.
- **Modularity** — Layers can be updated independently.
- **Maintainability** — Easier to debug and modify.

- Stability — Kernel stability is not affected by user-level programs.

Q3. Explain internal and external commands in UNIX. Write characteristics of UNIX OS.

★★ MEDIUM PRIORITY

Internal Commands: Built into the shell itself. No separate file exists on disk. They execute faster because no new process is created. Examples: `cd`, `echo`, `pwd`, `set`, `export`, `alias`, `exit`

External Commands: Stored as separate executable files on disk (usually in `/bin`, `/usr/bin`). A new child process is created to execute them. Examples: `ls`, `cp`, `mv`, `grep`, `cat`, `who`, `ps`

Characteristics of UNIX OS:

- Multiuser — Multiple users can use UNIX simultaneously.
- Multitasking — Multiple processes can run at the same time.
- Portability — Written mostly in C, easily portable.
- Hierarchical File System — Tree-structured directory system.
- Security — File permissions (read/write/execute) for owner, group, others.
- Pipes and Filters — Output of one command can be input to another.
- Shell Scripting — Powerful scripting capability.
- Open Source Variants — Linux, BSD are open-source UNIX-like systems.

Q4. What do the terms UID, PID and PPID stand for? For a currently executing process, how can you obtain those values?

★★ MEDIUM PRIORITY

UID (User ID): A unique numeric identifier assigned to each user in UNIX. Determines the user's access rights.

PID (Process ID): A unique numeric identifier assigned by the OS to each running process.

PPID (Parent Process ID): The PID of the parent process that created (forked) the current process.

How to obtain these values:

```
$ echo $$ # PID of current shell
$ echo $PPID # PPID of current shell
$ id # UID and GID of current user
$ ps -f # Full listing with PID, PPID, UID
$ who am i # Current user info
```

Q5. What is the type of kernel that gets completely copied to the primary memory while booting? (Ans: Monolithic Kernel)

★★ MEDIUM PRIORITY

Monolithic Kernel: A monolithic kernel is one where the entire OS kernel — including device drivers, file system management, memory management, and process scheduling — is compiled into a single large binary and loaded completely into primary memory (RAM) during booting.

Characteristics:

- All kernel services run in kernel space (privileged mode).
- Fast execution because no context switching between kernel modules.
- If one part crashes, the whole kernel may crash.
- Examples: Traditional UNIX, Linux kernel.

Other kernel types: Microkernel (only core services in kernel space — e.g., MINIX), Hybrid Kernel (Windows NT, macOS XNU).

Q6. UNIX supports multi-user and multitasking. Explain. What is the role of the shell?

★★ MEDIUM PRIORITY

Multi-user: UNIX allows multiple users to log in and use the system simultaneously. Each user has a unique UID, home directory, and set of permissions. The kernel manages resource sharing among all users.

Multitasking: UNIX can run multiple processes at the same time. The CPU scheduler uses time-slicing to switch among processes. A user can run processes in the background using & (e.g., `$ sort file &`).

Role of the Shell:

- **Command Interpreter** — Reads and interprets commands typed by the user.
- **Scripting** — Allows writing shell scripts (batch programs).
- **Interface** — Acts as an interface between the user and the kernel.
- **Types:** sh (Bourne), csh (C Shell), ksh (Korn), bash (Bourne Again Shell).
- UNIX shell is both a scripting language interpreter and a command interpreter.

Unit 2 — UNIX File System, Permissions & Links

Q7. What are the different types of files available in UNIX?

★★★ HIGH PRIORITY

UNIX treats everything as a file. The different types of files are:

- **Regular File (-):** Ordinary files containing text or binary data. E.g., program files, documents. Created with `touch`, `cat`, `vi`.
- **Directory File (d):** A file that contains a list of other files and directories. E.g., `/home`, `/bin`.
- **Symbolic Link (l):** A shortcut (pointer) to another file. Created with `ln -s source dest`.
- **Device File (b or c):** Represents hardware devices. Block device (b) — e.g., hard disk. Character device (c) — e.g., keyboard, terminal.
- **Named Pipe / FIFO (p):** Used for inter-process communication. Created with `mkfifo`.
- **Socket (s):** Used for network communication between processes.

Use `ls -l` to identify file type from the first character of the permission string.

Q8. Explain the structure of UNIX file system. Describe different kinds of files in UNIX.

★★★ HIGH PRIORITY

UNIX File System Structure: UNIX uses a hierarchical (tree-structured) file system with a single root directory `/`.

- **Boot Block:** Contains the bootstrap loader that loads the OS kernel during startup.
- **Super Block:** Contains metadata about the file system — total size, number of free blocks, number of i-nodes, file system state.
- **i-node Table:** Each file has one i-node storing file attributes (size, permissions, owner, timestamps, data block pointers). Does NOT store file name.
- **Data Blocks:** Actual file content is stored here.

Key Directories:

`/` Root directory

`/bin` Essential user commands (`ls`, `cp`, `mv`)

`/etc` Configuration files (`passwd`, `group`)

```
/home User home directories
/tmp Temporary files
/dev Device files
/usr User utilities and applications
/var Variable data (logs, mail)
```

Q9. Explain the significance of file permissions for a directory. What is umask? How can file permissions be defined with it? ★★★ HIGH PRIORITY

File Permissions in UNIX: Every file has 3 sets of permissions for: Owner (u), Group (g), Others (o). Each set has Read (r=4), Write (w=2), Execute (x=1).

Significance for a Directory:

- Read (r) — Allows listing of directory contents (ls).
- Write (w) — Allows creating or deleting files inside the directory.
- Execute (x) — Allows entering (cd) the directory and accessing its files.

Note: Directory permission is more important than file permission. Even if a file has full permissions, without execute permission on its parent directory, it cannot be accessed.

umask (User File Creation Mask): umask sets default permissions for newly created files and directories by masking (subtracting) bits from the maximum permission.

Default max: Files = 666, Directories = 777

```
If umask = 022:
File permission = 666 - 022 = 644 (rw-r--r--)
Directory permission = 777 - 022 = 755 (rwxr-xr-x)

$ umask # View current umask
$ umask 022 # Set umask to 022
```

Q10. What command gives read, write and execute to owner; read & execute to group; read only to others for file 'mid'? (\$chmod 754 mid) ★★★ HIGH PRIORITY

chmod command is used to change file permissions. It can use symbolic or absolute (octal) mode.

Octal values: Read=4, Write=2, Execute=1. Add them for each category.

```
Owner (u): r+w+x = 4+2+1 = 7
Group (g): r+x = 4+0+1 = 5
Others (o): r = 4+0+0 = 4

$ chmod 754 mid
```

Verification:

```
$ ls -l mid
-rwxr-xr-- 1 user group 0 Jun 20 mid
```

Symbolic mode equivalent:

```
$ chmod u=rwx,g=rx,o=r mid
```

Q11. Describe link and symbolic link. What is a race condition? Differentiate hard link and soft link. ★★ MEDIUM PRIORITY

Hard Link: A hard link creates another directory entry pointing to the same i-node as the original file. Both names refer to the same data.

```
$ ln file1 file2 # Hard link
```

- Same i-node number as original.
- Cannot span across file systems.
- Cannot link directories.
- File data survives even if original is deleted.

Soft (Symbolic) Link: A symbolic link is a special file that contains the path of another file. It is a pointer/shortcut.

```
$ ln -s file1 file2 # Symbolic link
```

- Has different i-node from original.
- Can span across file systems.
- Can link directories.
- Becomes a dangling link if original is deleted.

Race Condition: A race condition occurs when two or more processes access shared data simultaneously and the output depends on the order of execution. In file systems, if two processes try to create/delete the same file at the same time, a race condition may occur leading to unpredictable results.

Q12. Differentiate b/w absolute path and relative path with examples.

★★ MEDIUM PRIORITY

Absolute Path: The complete path from the root directory / to the file or directory. It always starts with /. It is independent of the current directory.

```
Example: /home/user/Documents/report.txt
```

```
/usr/bin/grep
```

Relative Path: The path relative to the current working directory. Does NOT start with /. Uses . (current dir) and .. (parent dir).

```
Example (if current dir is /home/user):
```

```
Documents/report.txt # relative
```

```
../anotheruser/file # go up one level
```

```
./script.sh # current directory
```

Key difference: Absolute path is same from anywhere in the system. Relative path changes based on current working directory.

Q13. What are the different blocks in UNIX file system? What does the superblock contain?

★★ MEDIUM PRIORITY

Blocks in UNIX File System:

- **Boot Block (Block 0):** First block of the file system. Contains bootstrap loader to start the OS. Present only in the root file system.
- **Super Block (Block 1):** Contains overall information about the file system.
- **i-node Table:** Array of i-nodes, one per file. Each i-node is 64 bytes and stores file metadata.
- **Data Blocks:** Actual file content stored here. A file may use direct, single-indirect, double-indirect, and triple-indirect block pointers.

Superblock contains:

- Total size of the file system (in blocks).
- Number of free blocks available.
- Number of free i-nodes.
- Size of i-node table.

- File system state (clean/dirty).
- Date of last update.
- Maximum number of files it can accommodate.

Q14. What is i-node? What are the various types of information stored in it?

★★ MEDIUM PRIORITY

i-node (Index Node): An i-node is a data structure in the UNIX file system that stores all metadata about a file except its name and actual data. Every file has exactly one i-node. i-nodes are stored in the i-node table.

Information stored in i-node:

- File type (regular, directory, device, etc.)
- File permissions (read/write/execute for owner, group, others)
- Owner's UID (User ID)
- Owner's GID (Group ID)
- File size (in bytes)
- Number of hard links to the file
- Timestamps: last access time, last modification time, last i-node change time
- Pointers to data blocks (direct, single-indirect, double-indirect, triple-indirect)

Note: File name is NOT stored in i-node. It is stored in the directory entry which maps file name → i-node number.

```
$ ls -li filename # View i-node number
```

```
$ stat filename # View all i-node information
```

Q15. How does UNIX access a file from a directory? Explain with example.

★★ MEDIUM PRIORITY

When a user accesses a file like `/home/user/file.txt`, UNIX follows these steps:

1. Start at root directory `/` (i-node 2, always fixed).
2. Read root directory's data block → find entry for 'home' → get its i-node number.
3. Read 'home' i-node → get data block of `/home` → find entry for 'user' → get i-node.
4. Read 'user' i-node → get data block of `/home/user` → find 'file.txt' → get i-node.
5. Read 'file.txt' i-node → get data block pointers → read actual file content.

Example:

```
Path: /home/user/file.txt
```

```
/ → i-node 2 → data block → 'home' entry (i-node 150)
```

```
home → i-node 150 → data block → 'user' entry (i-node 280)
```

```
user → i-node 280 → data block → 'file.txt' entry (i-node 450)
```

```
file.txt → i-node 450 → data blocks with actual content
```

Q16. Write the two ways to change permissions. 'Directory permission is more important than file permission.' Explain.

★★★ HIGH PRIORITY

Two ways to change file permissions using `chmod`:

1. Absolute (Octal/Numeric) Mode:

```
$ chmod 755 filename # rwxr-xr-x
```

```
$ chmod 644 filename # rw-r--r--
```

```
$ chmod 777 filename # rwxrwxrwx
```

2. Symbolic Mode:

```
$ chmod u+x filename # Add execute for owner  
$ chmod g-w filename # Remove write for group  
$ chmod o=r filename # Set others to read only  
$ chmod a+r filename # Add read for all
```

'Directory permission is more important than file permission':

Even if a file has full permissions (777), if the parent directory does not have execute (x) permission, you cannot access the file.

Example:

```
$ chmod a-rwx mydir # Remove all permissions on dir  
$ chmod 777 mydir/file.txt # File has full permissions  
$ cat mydir/file.txt # ERROR: Permission denied
```

- Without execute on directory → cannot cd into it.
- Without read on directory → cannot ls its contents.
- Without write on directory → cannot create/delete files in it.

Unit 3 — Process Management, Commands & Daemons

Q17. What is Zombie Process? When does it occur? Define Orphan process.

★★★ HIGH PRIORITY

Zombie Process: A zombie process (also called defunct process) is a process that has completed execution but still has an entry in the process table because its parent has not yet read its exit status using `wait()`.

When does it occur?

- Every process sends an exit status to its parent when it terminates.
- If the parent process does not call `wait()` to collect the exit status, the child remains as a zombie.
- The zombie holds its PID and exit status in the process table until the parent reads it.
- Too many zombies can exhaust the process table.

```
$ ps aux | grep Z # View zombie processes
```

Orphan Process: An orphan process is a process whose parent has terminated before the child. The orphan is adopted by the `init` process (PID=1), which periodically calls `wait()` to clean up.

Difference: Zombie — child finished, parent alive but not calling `wait()`. Orphan — parent finished, child still running (adopted by `init`).

Q18. How is a process created? Mention the role of fork and exec. System calls: Fork, Exec, Wait.

★★★ HIGH PRIORITY

Process Creation in UNIX: A new process is created using the `fork()` system call. UNIX uses the parent-child model.

`fork()` system call:

- Creates a copy (child process) of the calling (parent) process.
- Returns child's PID to parent, and 0 to the child.

- Both processes continue from the next instruction after fork().

```
pid = fork();
if (pid == 0) { /* child process */ }
else { /* parent process */ }
```

exec() system call:

- Replaces the current process image with a new program.
- The child process after fork() calls exec() to run a different program.
- Variants: execl(), execv(), execvp(), execl().

wait() system call:

- Parent calls wait() to pause until child finishes.
- Collects exit status of child — prevents zombie processes.

Complete flow: Parent calls fork() → child created → child calls exec() to run new program → parent calls wait() → child exits → parent resumes.

Q19. Define: sticky bit, daemon, nice command, touch command, nohup, PID vs PPID, Process State Diagram, init process.

★★★ HIGH PRIORITY

Sticky Bit: When set on a directory, only the file owner, directory owner, or root can delete/rename files. E.g., /tmp has sticky bit set. Command: chmod +t /tmp. Shows as t in ls -l.

Daemon: A background process that runs continuously to perform system services. Not connected to any terminal. Examples: httpd (web server), cron (scheduler), sshd (SSH). Started at boot time.

nice command: Used to set the priority of a process. Priority range: -20 (highest) to +19 (lowest). Default nice value = 0.

```
$ nice -n 10 command # Lower priority
$ renice -5 PID # Change priority of running process
```

touch command: Creates an empty file if it doesn't exist. Updates timestamps (access and modification time) of an existing file.

```
$ touch filename # Create or update timestamp
```

nohup: Runs a command immune to hangup (HUP) signals. The process continues even after the user logs out. Output goes to nohup.out.

```
$ nohup command &
```

PID vs PPID: PID = Process ID (unique ID of process). PPID = Parent Process ID (PID of parent that created this process).

Process States: New → Ready → Running → Blocked/Waiting → Terminated. A process moves between states based on CPU scheduling and I/O operations.

init process: The first process started by the kernel (PID=1). It is the ancestor of all processes. Reads /etc/inittab to start system services. Adopts orphan processes.

Q20. Explain the use of following UNIX commands: mv, cp, ps, top, du, grep, sed, tar, nohup, who, wc, head, tail.

★★ MEDIUM PRIORITY

- **mv** — Move or rename files/directories. \$ mv file1 file2
- **cp** — Copy files. \$ cp file1 file2 | \$ cp -r dir1 dir2
- **ps** — Display running processes. \$ ps -ef (all processes full format).

- **top** — Real-time view of processes, CPU and memory usage. Press q to quit.
- **du** — Disk usage of files/directories. `$ du -sh *`
- **grep** — Search for pattern in file. `$ grep 'pattern' file`
- **sed** — Stream editor for find/replace. `$ sed 's/old/new/g' file`
- **tar** — Archive files. `$ tar -cvf archive.tar files | $ tar -xvf archive.tar`
- **nohup** — Run command after logout. `$ nohup cmd &`
- **who** — Show logged-in users. `$ who | $ who am i`
- **wc** — Count lines, words, characters. `$ wc -l file (lines) | $ wc -w file (words)`
- **head** — Display first N lines. `$ head -10 file`
- **tail** — Display last N lines. `$ tail -10 file | $ tail -f logfile (follow)`

Q21. What is filter? Give example. (grep, sed, AWK)

★★★ HIGH PRIORITY

Filter: A filter is a program that takes its input from the standard input (stdin), performs some operation on that input, and writes the results to the standard output (stdout). Filters are typically used with pipes.

Commonly used filters in UNIX:

1. **grep** — Global Regular Expression Print. Searches for a pattern in a file and prints matching lines.

```
$ grep 'apple' file.txt
$ grep -c 'abc' sample.txt # Count occurrences
$ cat f1 f2 | grep 'pattern' # Use with pipe
```

2. **sed** — Stream Editor. Performs text transformations on a stream of text.

```
$ sed 's/old/new/g' file.txt # Replace all
$ sed '2d' file.txt # Delete line 2
```

3. **AWK** — Named after Aho, Weinberger, Kernighan. Powerful for pattern scanning and data extraction.

```
$ awk '{print $1}' file.txt # Print 1st column
$ awk '/pattern/{print}' file # Print matching lines
```

Example using pipe:

```
$ cat apple.txt | wc
Output: 3 4 21 (lines words characters)
```

Q22. What are utilities of environment variables? Differentiate hard links and symbolic links.

★★ MEDIUM PRIORITY

Environment Variables: Variables that store configuration information for the shell and programs.

- **PATH** — Directories to search for commands.
- **HOME** — Current user's home directory.
- **TERM** — Terminal type.
- **PS1** — Primary prompt string (e.g., \$ or #).
- **PS2** — Secondary prompt (continuation, default >).
- **SHELL** — Path of current shell.
- **USER** — Current logged-in username.

```
$ echo $HOME # Print HOME variable
$ export VAR=value # Set environment variable
```

Hard Link vs Symbolic Link:

Feature	Hard Link	Symbolic Link
i-node	Same as original	Different i-node
Cross FS	No	Yes
Link dirs	No	Yes
If orig deleted	File still exists	Dangling link
Created by	ln file link	ln -s file link

Q23. Short Note: Kernel, Shell, System Calls, Process State.

★★ MEDIUM PRIORITY

Kernel: The core component of the OS that directly manages hardware resources. Responsibilities: process scheduling, memory management, file system management, device drivers, inter-process communication. Runs in privileged (kernel) mode.

Shell: The command-line interpreter that acts as an interface between the user and the kernel. It reads user commands, interprets them, and sends them to the kernel. Types: sh, csh, ksh, bash. Shell is also a scripting language.

System Calls: The mechanism by which a user-space program requests services from the kernel. They provide a controlled interface to hardware and OS services. Examples: fork(), exec(), wait(), open(), read(), write(), close(), exit().

Process States:

- **New:** Process is being created.
- **Ready:** Process is waiting to be assigned to CPU.
- **Running:** Instructions are being executed by CPU.
- **Blocked/Waiting:** Process is waiting for I/O or event.
- **Terminated/Zombie:** Process has finished execution.

Unit 4 — Vi Editor, grep / egrep / fgrep & Escape Sequences

Q24. What is the difference b/w Vi commands: :w, :q!, :wq, ZZ? Write short note on Ex mode operations of Vi.

★★★ HIGH PRIORITY

Vi Editor Commands (Ex mode):

:w	Save (write) the file without quitting.
:q!	Quit without saving. Discards all changes.
:wq	Save the file and quit vi.
ZZ	Save and quit (same as :wq, in command mode).
:w file	Save to a new file name.
:q	Quit (only if no unsaved changes).
:set nu	Show line numbers.
:N	Go to line N (e.g., :10 goes to line 10).
:/pattern	Search for pattern forward.
:s/old/new	Replace first occurrence on current line.
:%s/old/new/g	Replace all occurrences in file.

Ex Mode: Ex mode is accessed by pressing : from command mode. It allows file operations (save, quit), search-replace, line operations, and setting editor options. It is the most powerful mode for file manipulation in Vi.

Q25. Write short note on Vi editor. Three modes of Vi. Which is the default mode?

★★★ HIGH PRIORITY

Vi Editor: Vi (Visual Editor) is a screen-based text editor available on all UNIX systems. It was created by Bill Joy. It is efficient for editing files on remote servers or minimal systems. Vim (Vi iMproved) is an enhanced version.

Three Modes of Vi:

- **1. Command Mode (Normal Mode) — DEFAULT MODE:** Vi starts in this mode. Keys perform editing commands (delete, copy, paste, navigate). You cannot type text directly. Press Esc to return here.

dd – Delete current line

yy – Copy (yank) current line

p – Paste

/pattern – Search

nG – Go to line n

- **2. Insert Mode:** Allows typing/inserting text. Entered by pressing:

i – Insert before cursor

a – Append after cursor

o – Open new line below

I – Insert at beginning of line

A – Append at end of line

- **3. Ex Mode (Last Line Mode):** Accessed by pressing :. Used for file operations and global commands (:w, :q, :s, :set).

Default mode: Command Mode. Vi always starts in Command mode.

Q26. Explain meaning of grep, egrep, fgrep with differences.

★★★ HIGH PRIORITY

grep (Global Regular Expression Print): Searches for a Basic Regular Expression (BRE) pattern in files and prints matching lines. It is the standard pattern matching tool.

```
$ grep 'pattern' file.txt
```

```
$ grep -i 'error' logfile # Case-insensitive
```

```
$ grep -n 'word' file # With line numbers
```

```
$ grep -v 'pattern' file # Invert match
```

```
$ grep -c 'word' file # Count matches
```

egrep (Extended grep): Uses Extended Regular Expressions (ERE). Supports +, ?, |, (), {} without backslashes. Slower than fgrep but more powerful than grep.

```
$ egrep 'cat|dog' file # Match cat OR dog
```

```
$ egrep 'go+gle' file # One or more 'o'
```

fgrep (Fixed grep / Fast grep): Searches for fixed strings (no regular expressions). Fastest of the three because no regex processing. Treats special characters literally.

```
$ fgrep 'hello.world' file # '.' treated literally
```

Comparison:

```
grep – Basic regex (BRE) – Medium speed
```

```
egrep – Extended regex (ERE) – Slowest (most powerful)
```

```
fgrep – Fixed string – Fastest (no regex)
```

Q27. What is escape sequence? Give example.

★★ MEDIUM PRIORITY

Escape Sequence: An escape sequence is a sequence of characters that does not represent itself in a string but is translated into another character or command. In UNIX shell and C, they begin with a backslash \.

Common Escape Sequences:

\n	– Newline (move to next line)
\t	– Horizontal Tab
\r	– Carriage Return
\b	– Backspace
\\	– Literal backslash
\'	– Single quote
\"	– Double quote
\0	– Null character

Example in shell:

```
$ echo -e "Hello\nWorld"
Hello
World

$ echo -e "Name:\tSubhadra"
Name: Subhadra
```

Q28. grep full form. TCP vs UDP — connection-oriented vs connectionless protocol.

★★ MEDIUM PRIORITY

grep stands for: **G**lobal **R**egular **E**xpression **P**rint.

TCP (Transmission Control Protocol):

- Connection-oriented protocol.
- Establishes a connection before data transfer (3-way handshake).
- Guarantees delivery, ordering, and error checking.
- Slower but reliable. Used for HTTP, FTP, email.

UDP (User Datagram Protocol):

- Connectionless protocol.
- No connection established before sending data.
- No guarantee of delivery or ordering.
- Faster but unreliable. Used for DNS, video streaming, gaming.

In UNIX, socket files (type 's' in ls -l) are used for TCP/UDP inter-process communication. Pipe uses — only direct block; TCP is connection-oriented, UDP is connectionless.

Unit 5 — Shell Scripting & Programming

Q29. Types of shell and its features. Write meaning of 'command interpreter'. Why shell is called 'command interpreter'?

★★★ HIGH PRIORITY

Types of Shell:

- **Bourne Shell (sh):** Original UNIX shell by Stephen Bourne. Simple and fast. Default in many systems.
- **C Shell (csh):** C-like syntax. Has history and aliasing. Created by Bill Joy.
- **Korn Shell (ksh):** Combines features of sh and csh. More powerful scripting.

- **Bash (Bourne Again Shell):** Most popular. Default in Linux. Combines best of sh and csh. Supports arrays, arithmetic.
- **Z Shell (zsh):** Advanced shell with better auto-complete and customization.

Command Interpreter: A command interpreter reads commands entered by the user, interprets (parses) them, and passes them to the OS for execution. It translates human-readable commands into actions the kernel can perform.

Why shell is called command interpreter:

- It reads each command from the user (or script).
- Parses the command — identifies program name, arguments, options.
- Looks up the command in PATH and finds the executable.
- Creates a child process (fork) and executes the program (exec).
- Waits for it to finish, then shows the result.
- Shell scripts are interpreted line by line, not compiled.

Hence, shell is a scripting language interpreter AND a command interpreter.

Q30. What do you mean by positional parameter? Shell parameters: \$\$, \$*, \$@, \$?, \$!. Give example.

★★★ HIGH PRIORITY

Positional Parameters: In shell scripts, arguments passed on the command line are accessed using positional parameters. \$1 is the first argument, \$2 is the second, and so on.

```
$ ./script.sh Hello World
$0 = script.sh (script name)
$1 = Hello
$2 = World
```

Special Shell Parameters:

```
$$ – PID of the current shell (running script)
$* – All positional parameters as a single string
$@ – All positional parameters as separate quoted strings
$? – Exit status of the last executed command (0=success)
$! – PID of the last background process
$# – Number of arguments supplied to the script
```

Example script:

```
#!/bin/bash
echo "Script name: $0"
echo "First arg: $1"
echo "All args: $*"
echo "Num args: $#"
```

Q31. Write shell scripts to check: Prime, Palindrome, Factorial, Fibonacci, Armstrong.

★★★ HIGH PRIORITY

1. Check Prime Number:

```
#!/bin/bash
echo "Enter number:"; read n
flag=0
```

```
for((i=2;i<n;i++))
do
if [ $((n%i)) -eq 0 ]; then flag=1; break; fi
done
if [ $flag -eq 0 ]; then echo "$n is Prime"
else echo "$n is Not Prime"; fi
```

2. Check Palindrome:

```
#!/bin/bash
echo "Enter string:"; read str
rev=$(echo $str | rev)
if [ "$str" = "$rev" ]; then echo "Palindrome"
else echo "Not Palindrome"; fi
```

3. Factorial:

```
#!/bin/bash
echo "Enter number:"; read n
fact=1
for((i=1;i<=n;i++)); do fact=$((fact*i)); done
echo "Factorial = $fact"
```

4. Fibonacci Series:

```
#!/bin/bash
echo "Enter terms:"; read n
a=0; b=1
echo -n "$a $b "
for((i=2;i<n;i++))
do c=$((a+b)); echo -n "$c "; a=$b; b=$c; done
echo
```

5. Armstrong Number:

```
#!/bin/bash
echo "Enter number:"; read n
temp=$n; sum=0
while [ $temp -ne 0 ]
do d=$((temp%10)); sum=$((sum+d*d*d)); temp=$((temp/10)); done
if [ $sum -eq $n ]; then echo "Armstrong"
else echo "Not Armstrong"; fi
```

Q32. How is copying a file different from linking it? Give suitable examples.

★★ MEDIUM PRIORITY

Copying a file (cp): Creates an entirely new independent copy of the file with a new i-node and new data blocks. Changes to the copy do not affect the original.

```
$ cp file1.txt file2.txt
```

- New i-node created for file2.txt.
- New data blocks allocated — data duplicated.
- Consumes extra disk space.
- Independent — modifying one doesn't affect the other.

Linking a file (ln):

Hard Link — Points to same i-node. No data duplication. Both names share the same data.

```
$ ln file1.txt file3.txt
```

- Same i-node as file1.txt.
- No extra disk space for data.
- Changes in one are reflected in the other.

Soft Link — Points to the path of original. Separate i-node.

```
$ ln -s file1.txt file4.txt
```

- New i-node; contains path of file1.txt.
- If original deleted, soft link becomes invalid (dangling).

Q33. Shell parameters — \$1 and \$!. Shell is used to protect from unsafe access to kernel space.

★★ MEDIUM PRIORITY

\$1 — Represents the first positional parameter (first command-line argument) passed to a shell script.

```
$ ./myscript.sh Hello
```

```
$1 = Hello
```

\$! — Stores the PID of the last background process started with &.

```
$ sleep 100 &
```

```
$ echo $! # Prints PID of sleep process
```

Why shell is used to protect from unsafe access to kernel space:

- User programs cannot directly call kernel functions or access hardware.
- The shell mediates all requests from users to the kernel through system calls.
- This prevents unauthorized or accidental access to kernel memory space.
- The shell enforces user permissions before executing privileged operations.
- A shell is used because it provides a safe, controlled interface between user space and kernel space.

This separation of user space and kernel space is fundamental to UNIX security and stability.